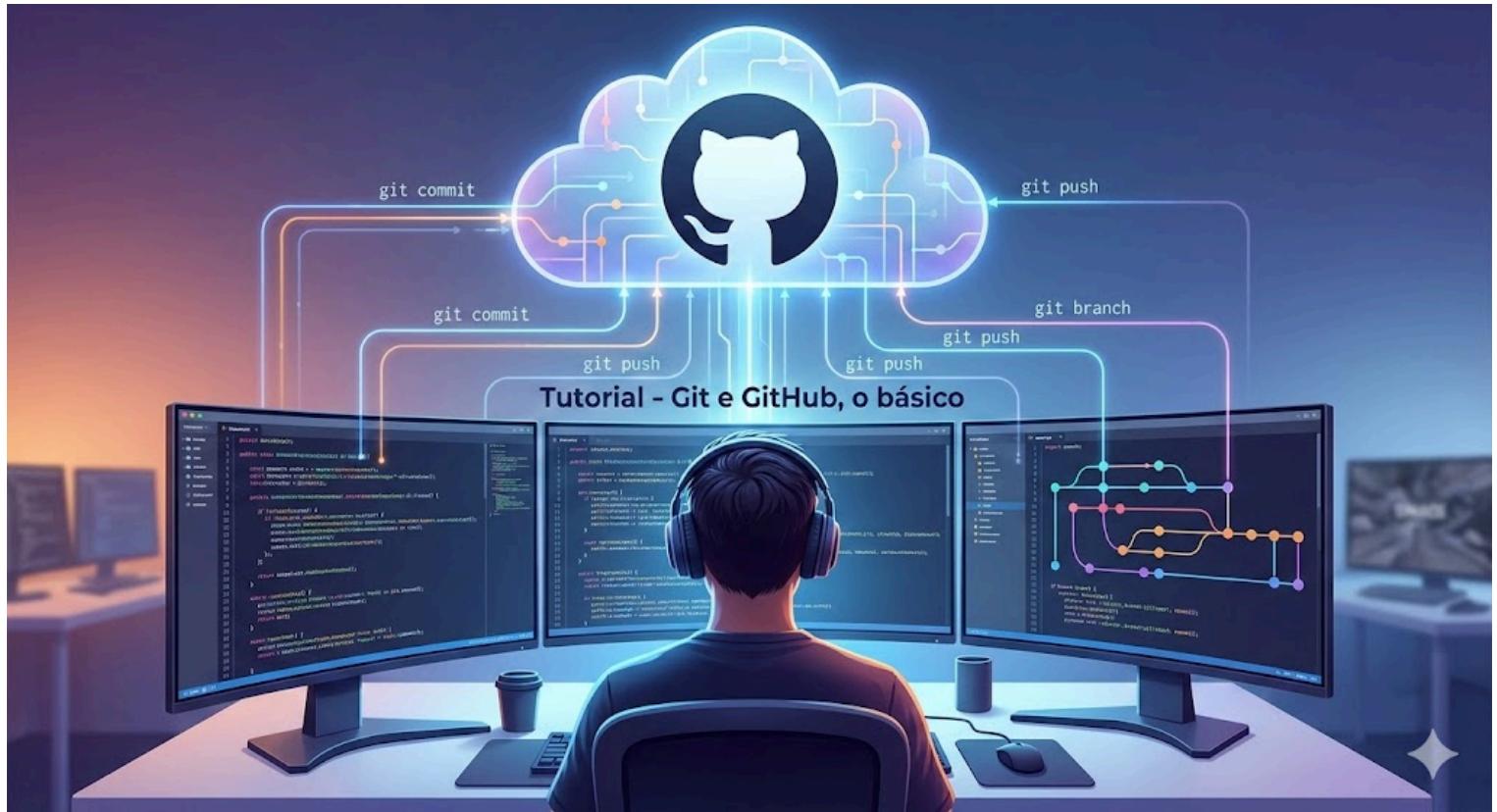


Git e GitHub: O Básico



🚀 Por que dominar Git e GitHub transforma sua carreira?

Ao longo da sua jornada, você vai escrever milhares de linhas de código. Sem um sistema de controle, esse processo torna-se:

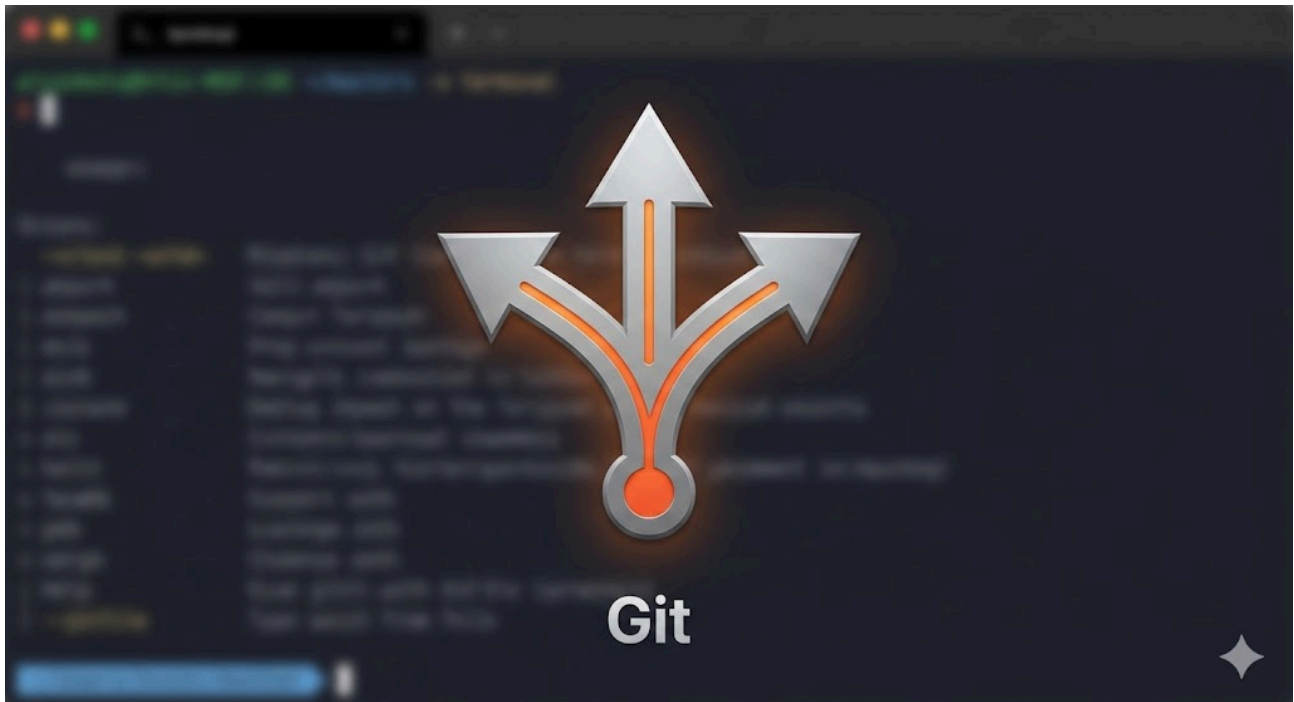
O Problema (Sem Versionamento)	A Solução (Com Git e GitHub)
🔴 Arriscado: Medo de quebrar o código.	🟢 Segurança: Experimente sem medo; reverta se algo der errado.
🌀 Caótico: Múltiplas cópias (projeto_final_v2.zip).	🟢 Histórico Completo: Cada passo é registrado de forma organizada.
🚧 Limitado: Difícil trabalhar em equipe.	🟢 Colaboração Profissional: Revisões, sugestões e integração suave.
👤 Isolado: Apenas seus projetos.	🟢 Autonomia Global: Contribua com projetos open source do mundo inteiro.

🔧 O Que São, Afinal?

Não confunda os dois. Pense neles como o **motor** e a **garagem**.

1. Git (O Sistema)

É um **sistema de controle de versão distribuído**, criado por Linus Torvalds em 2005. Ele permite que desenvolvedores **acompanhem, compartilhem e revertam** qualquer mudança no código com segurança e agilidade .



Características Principais:

-  **Distribuído:** Cada dev tem uma cópia completa do repositório.
-  **Rápido:** Operações como commit, log e branch são instantâneas.
-  **Seguro:** Cada alteração gera um hash único (SHA-1) para integridade.
-  **Flexível:** Suporta múltiplos fluxos e estratégias de trabalho.

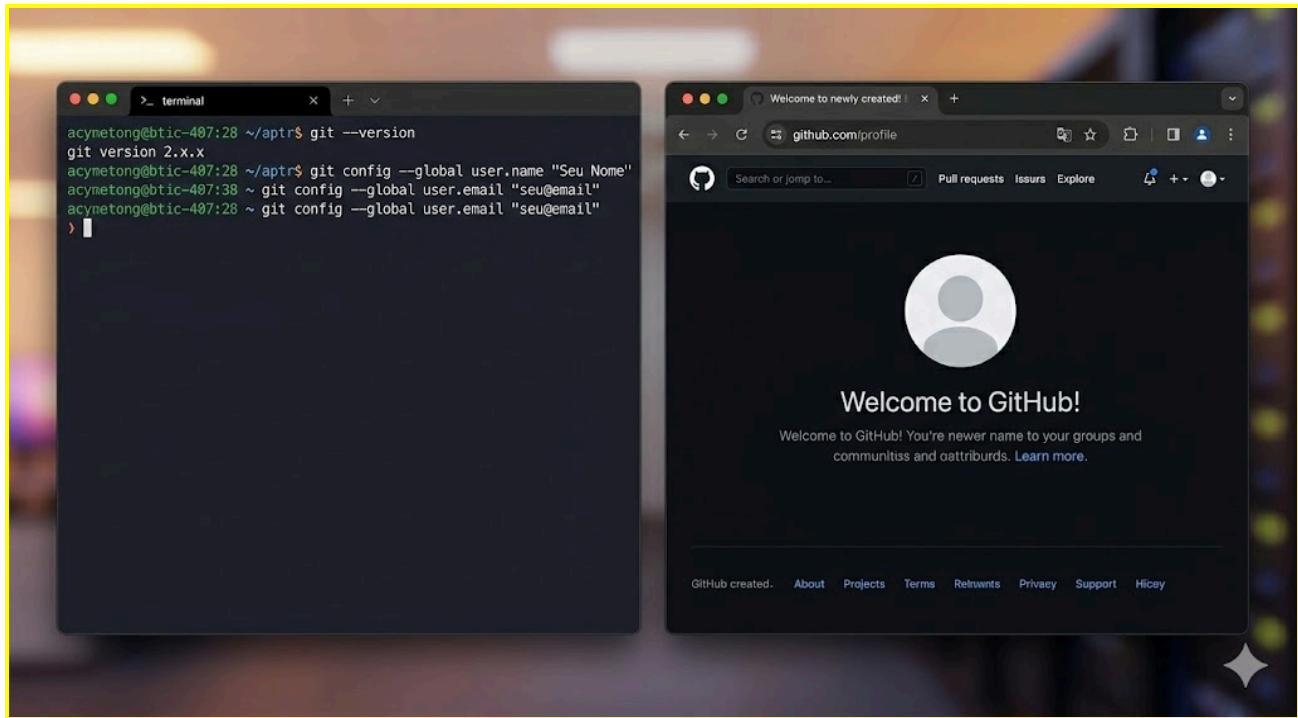
2. GitHub (A Plataforma)



É uma plataforma de hospedagem e colaboração online baseada em Git. Nele você pode:

-  **Criar e manter** repositórios online.
-  **Trabalhar em equipe** usando Pull Requests.
-  **Organizar tarefas** com o Issues.
-  **Automatizar fluxos** com o Actions.
-  **Conectar** com devs e empresas.

Instalação e Configuração Inicial



Passo 1: Instalação

-  **No Windows:** Acesse git-scm.com, baixe e execute o instalador padrão.
-  **No Linux (Debian/Ubuntu):** Utilize os comandos no terminal:

Bash

```
sudo apt update
```

```
sudo apt install git
```

```
git --version # Para verificar a instalação
```

Dica de Integração: O GitHub possui **integração nativa** com o **VSCode** através da extensão "**GitLens**". Com ela, você pode usar o terminal do VSCode para enviar **comandos Git**.

Passo 2: Configuração das Credenciais

Importante: O e-mail cadastrado na sua conta GitHub é o que dará acesso aos comandos Git.

1. **Crie sua conta:** Acesse github.com e crie sua conta.

2. **Configurar o Terminal:** Os comandos do Git serão todos no **terminal (Windows e Linux)**. Use os comandos abaixo para **"logar"** o **Git** no seu computador :

```
Bash
# Substitua pelos SEUS dados do GitHub
git config --global user.name "Seu Nome de Usuário"
git config --global user.email "seu@email.com"
```

3. **Confirmar Configuração:** Execute para verificar se deu certo:

```
Bash
git config user.name # Deve aparecer seu nome
git config user.email # Deve aparecer seu email
git config --list # Visualiza todas as configurações
```

 Os Comandos Essenciais (Sua Caixa de Ferramentas)

Regra de Ouro: Os comandos Git devem ser feitos SEMPRE dentro da pasta raiz do seu projeto (repositório local).

Para Iniciar um Projeto

Comando	Descrição	Quando usar?
git init	Cria um repositório vazio dentro de uma pasta no computador.	Para começar um projeto do zero no seu PC.
git clone (URL)	Copia um repositório existente no GitHub para seu computador local.	Quando você quer baixar um projeto que já está online.

Para o Controle de Versão Diário



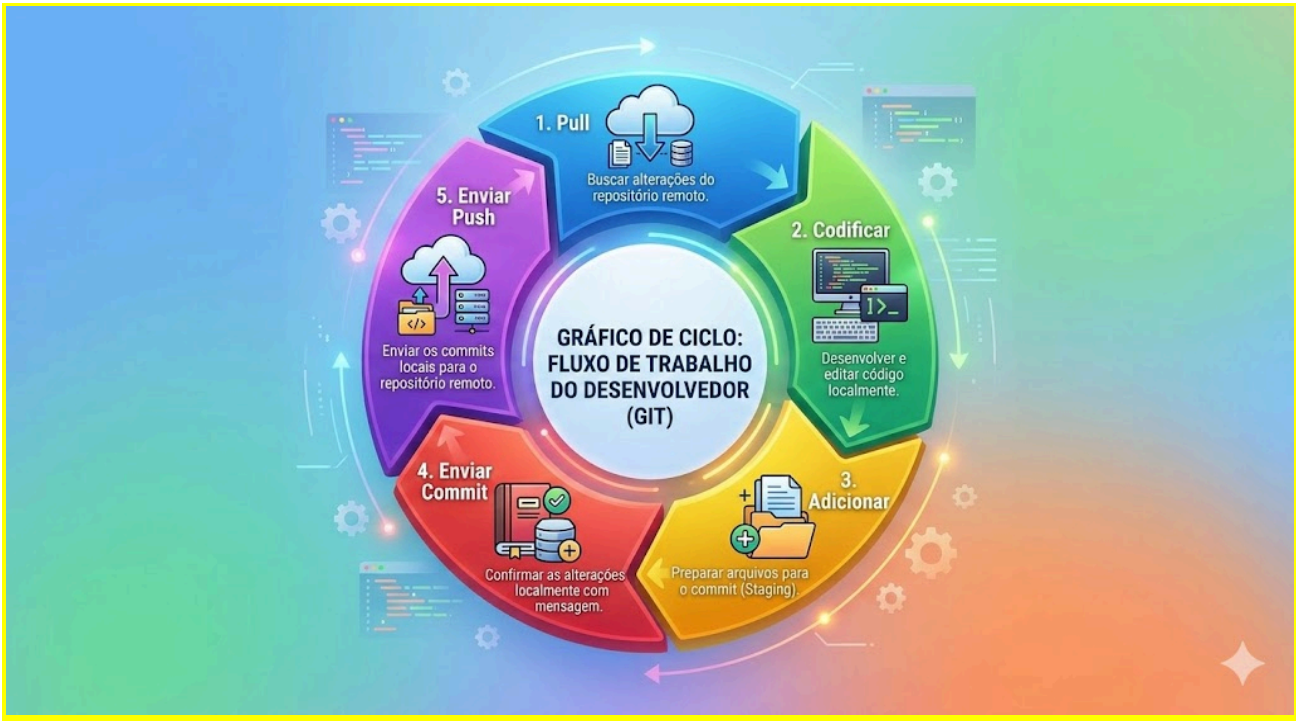
Comando	Descrição	Entenda o Fluxo
<code>git pull</code>	Recebe as atualizações do repositório remoto (GitHub) para o seu local.	Traga o que há de novo antes de começar.
<code>git add .</code>	Move todos os arquivos modificados para a área de preparação (Staging Area).	Preparando o pacote para envio.
<code>git add (nome)</code>	Move um arquivo específico para a área de preparação.	Use para selecionar apenas o que deseja enviar.
<code>git commit -m "mensagem"</code>	Registra o envio localmente, salvando o estado atual do código com uma breve descrição.	Informando <i>o que</i> foi feito nessa alteração.
<code>git push</code>	Envia os arquivos confirmados (commits) da área de espera local para o repositório online (GitHub).	Torne suas alterações visíveis para todos.

Comandos Especiais de Diagnóstico

- 🔍 `git status`: Mostra o status dos arquivos. Visualize quais arquivos foram modificados ou estão pendentes de commit.
- 📖 `git log`: Lista o histórico de commits, permitindo revisar o progresso e quem fez o quê.

🔄 Exemplo de Fluxo Diário Ideal

Siga esta ordem de comandos para trabalhar de forma segura no seu repositório local, evitando erros :



1.  **git pull**: **SEMPRE COMECE AQUI**. Ideal para trabalhar em equipes, garante que você está começando com a versão mais atualizada. Realize o "pull" *antes* de tentar fazer o "commit" para evitar conflitos no terminal.
2.  **(Codifique)**: Faça suas alterações nos arquivos.
3.  **git add** . (ou **git add arquivo**): Após finalizar, envie as alterações para a Staging Area.
4.  **git commit -m "Sua mensagem descritiva"**: Utilize linguagem simples e relevante (ex: "Corrige bug no login" ou "Adiciona botão de cadastro"). Isso garante a rastreabilidade.
5.  **git push**: Envie seu pacote de alterações para o GitHub. Agora seu material estará à disposição de todos os colaboradores.

Conclusão: Este tutorial é apenas o começo da jornada. O Git/GitHub é uma ferramenta essencial e poderosa para o desenvolvimento de sistemas. Pratique esses comandos básicos e, em breve, eles se tornarão automáticos!

